ARTIFICIAL INTELLIGENCE (Q) – APRIL 2026
KECERDASAN TIRUAN (Q) – APRIL 2026

ANT COLONY OPTIMIZATION

Optimasi Koloni Semut

Mohammad Iqbal, S.Si., M.Si., Ph. D.

OUTLINES

- Swarm Intelligence
- Introduction to Ant Colony Optimization
- Ant Behaviour
- Stigmergy
- Pheromones
- ACO Algorithm
- Advantages & Disadvantages

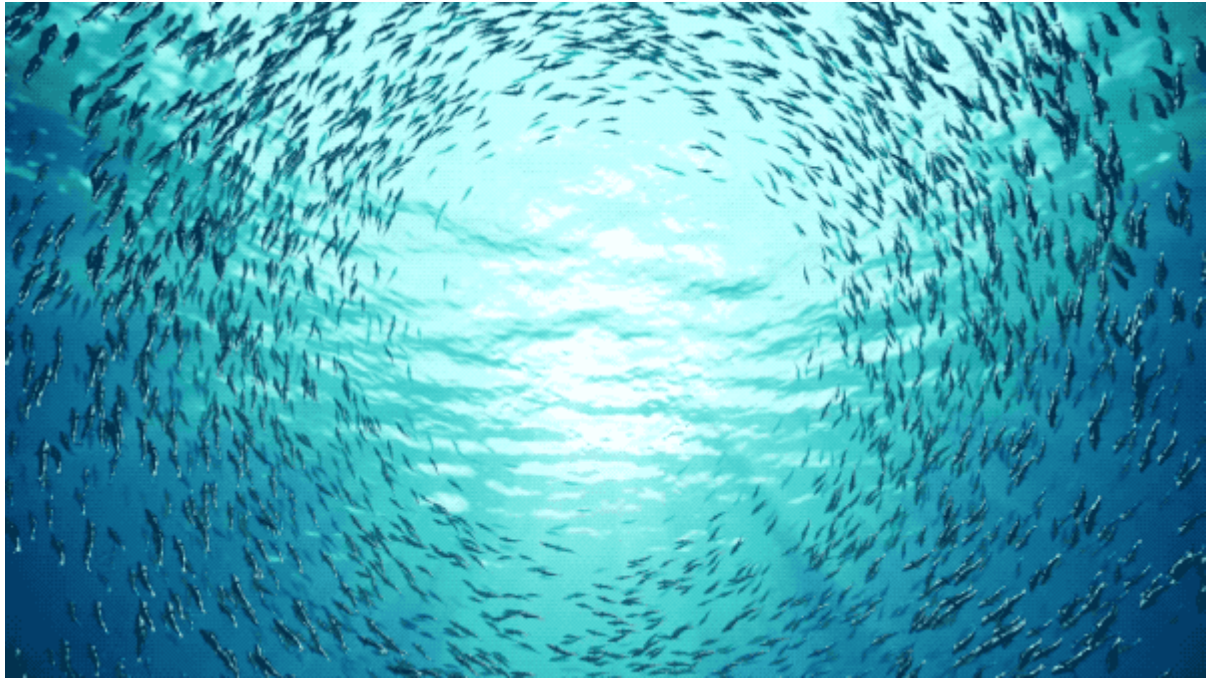
OUTLINES

Swarm Intelligence

Swarm Intelligence -

- Artificial intelligence technique based on the study of **collective behavior in decentralized, self-organized systems**
- Introduced by Beni & Wang in 1989
- **Collective system** capable of performing complex tasks in
- a dynamic environment
- Model suited to **distributed problem solving**
- Works without:
  - External guidance
  - Central coordination
- Typically made up of a population of **simple** agents

Swarm Intelligence - Fish



Swarm Intelligence - Ants

Swarm Intelligence - Bee

Swarm Intelligence - Komodo

OUTLINES

Ant Colony Optimization

ACO

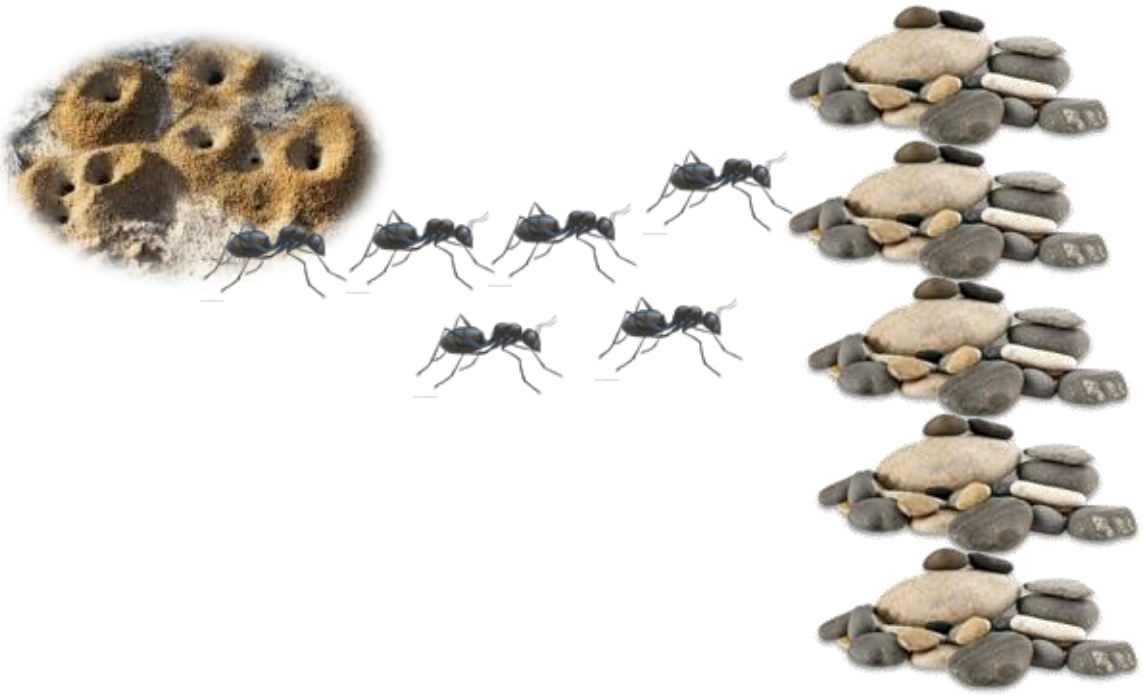
- Proposed by **Marco Dorigo** in 1991
- Inspired in the behavior of real ants
- **Multi-agent approach** for solving complex combinatorial
- optimization problems
- Applications:
  - Traveling Salesman Problem
  - Scheduling
  - Network Model Problem
  - Vehicle routing

Ant Behavior

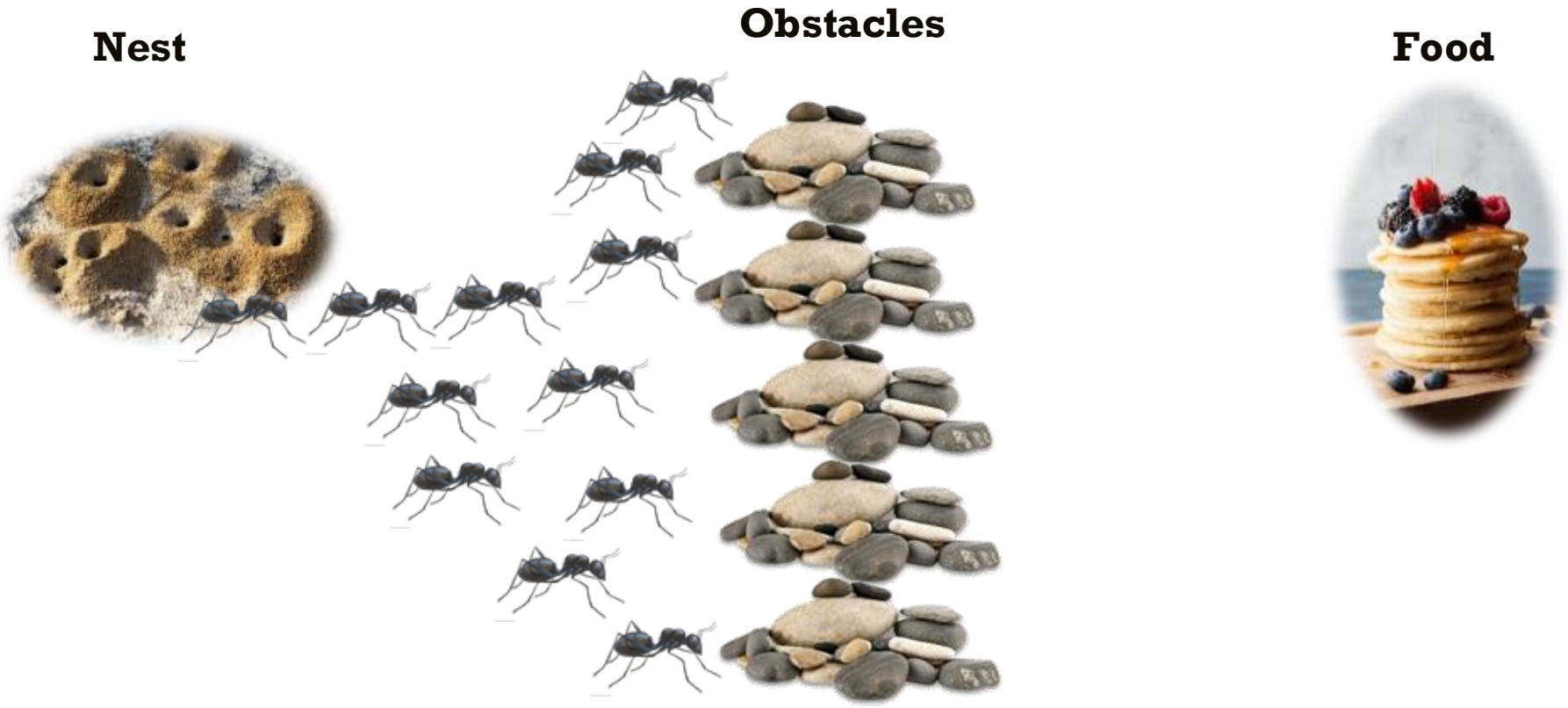
Nest

Obstacles

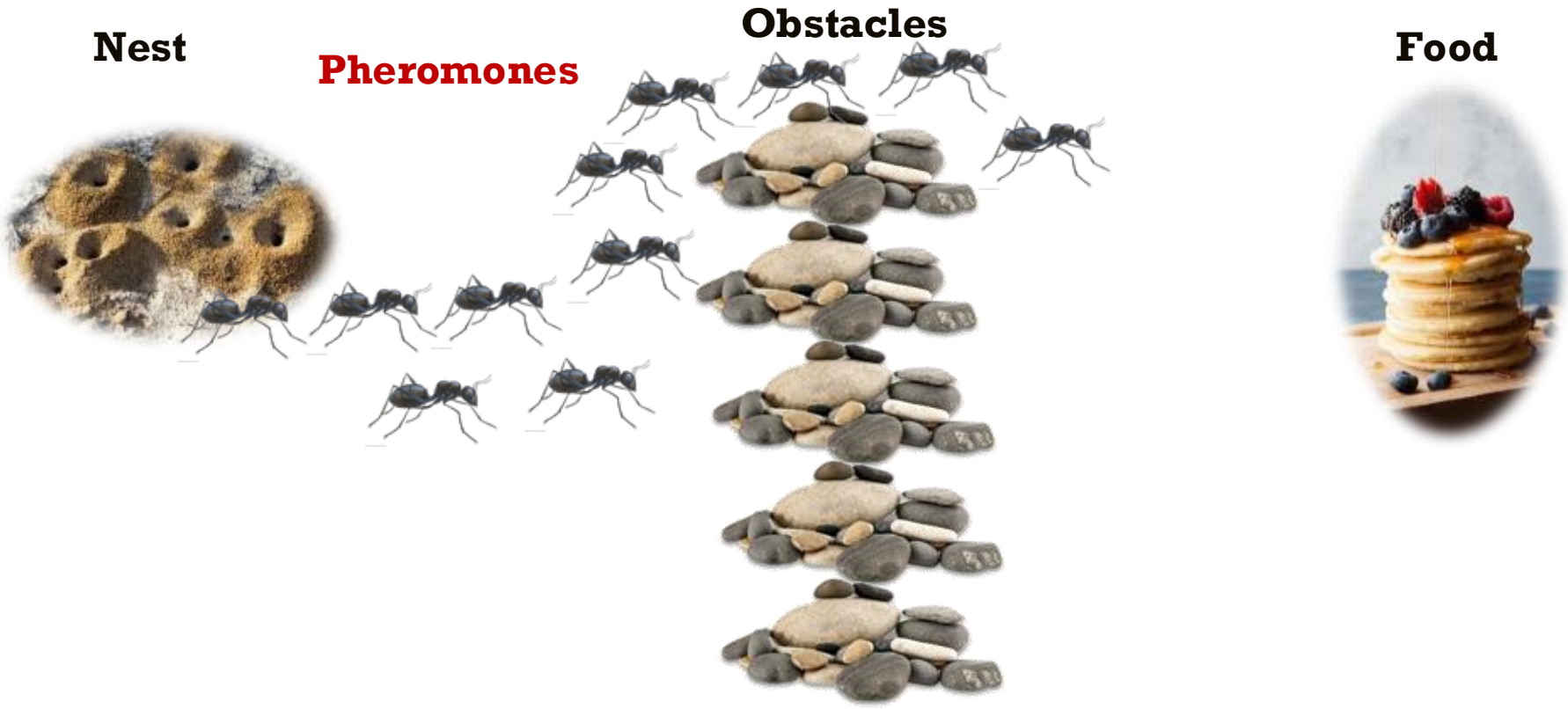
Food



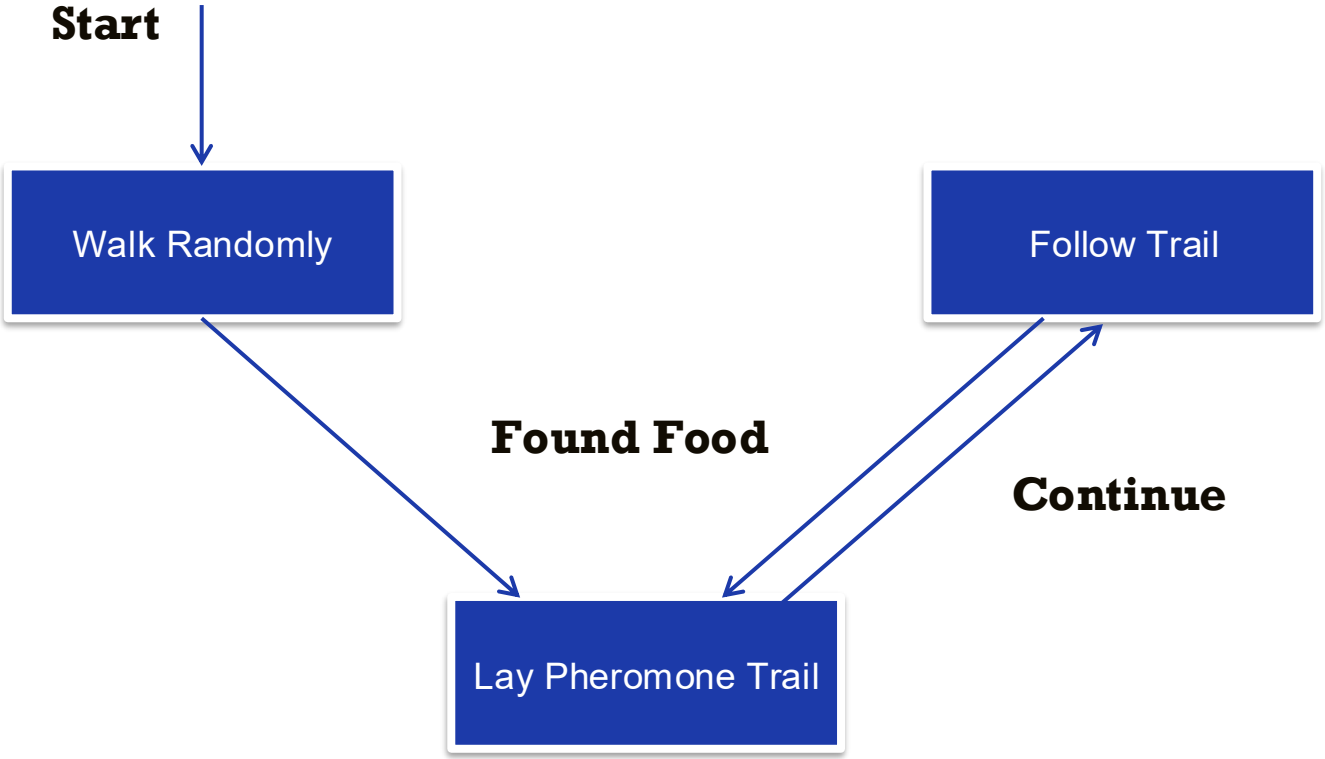
Ant Behavior



Ant Behavior



Ant Behavior (Quest for Food)



Ant Behavior

- Ant behavior is **stochastic**
- The behavior is induced by **indirect communication**
- (pheromone paths) - **Stigmergy**
- Ants explore the search space
- Limited ability to sense local environment
- Act **concurrently** and **independently**
- High quality solutions emerge via **global cooperation**

Stigmergy

- Term coined by French biologist Pierre-Paul Grasse, means
- interaction through the environment
- Indirect communication via **interaction with environment**
- Agents **respond to changes** in the environment
- Allows simpler agents
- Decreases direct communication

Pheromones

- Ants lay pheromone trails while traveling
- Pheromones **accumulate** with multiple ants using a path
- This behavior leads to the appearance of **shortest paths**

Pheromones = long-term memory of an ant colony

- Pheromones **evaporate**
  - **Avoid being trapped** in local optima

ρ small → low evaporation → **slow adaptation**

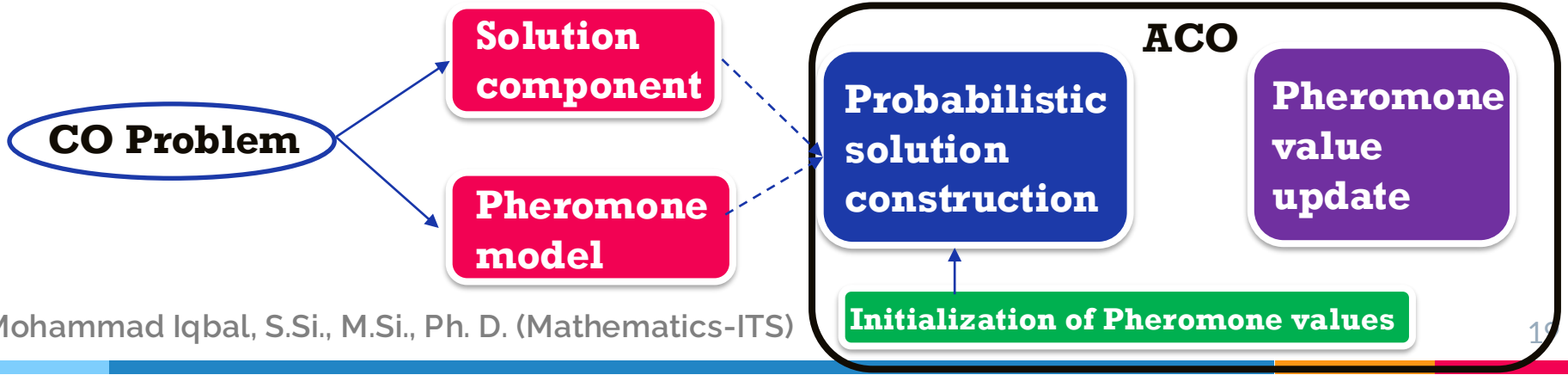
ρ large → high evaporation → **fast adaptation**

OUTLINES

Ant Colony Algorithm

ACO Algorithm

- **Construct solutions**
  - Explore the search space
  - Choose next step **probabilistically** according to the **pheromone model**
- Apply local search to constructed solutions (Optional)
- **Update pheromones** (add new + evaporate)



Simple ACO Algorithm

Algorithm 17.2 Simple ACO Algorithm

```
Initialize τ_ij(0) to small random values;
Let t = 0;
Place n_k ants on the origin node;
repeat
    for each ant k = 1, ..., n_k do
        //Construct a path x^k(t);
        x^k(t) = ∅;
        repeat
            Select next node based on the probability defined in equation (17.2);
            Add link (i, j) to path x^k(t);
        until destination node has been reached;
        Remove all loops from x^k(t);
        Calculate the path length f(x^k(t));
    end
    for each link (i, j) of the graph do
        //pheromone evaporation;
        Reduce the pheromone, τ_ij(t), using equation (17.5);
    end
    for each ant k = 1, ..., n_k do
        for each link (i, j) of x^k(t) do
            Δτ^k = 1/f(x^k(t));
            Update τ_ij using equation (17.4);
        end
    end
    t = t + 1;
until stopping condition is true;
Return the path x^k(t) with smallest f(x^k(t)) as the solution;
```

SACO Algorithm

Pheromone

$$p_{ij}^{k}(t) = \begin{cases} \dfrac{\tau_{ij}^{\alpha}(t)}{\sum_{j \in N_i^k(t)} \tau_{ij}^{\alpha}(t)} & \text{if } j \in N_i^k(t) \\ 0 & \text{if } j \notin N_i^k \end{cases}$$

$$\Delta\tau_{ij}^{k}(t) \propto \frac{1}{L^k(t)}$$

$$\tau_{ij}(t+1) = \tau_{ij}(t) + \sum_{k=1}^{n_k} \Delta\tau_{ij}^{k}(t)$$

Evaporation

$$\tau_{ij}(t) \leftarrow (1-\rho)\tau_{ij}(t)$$

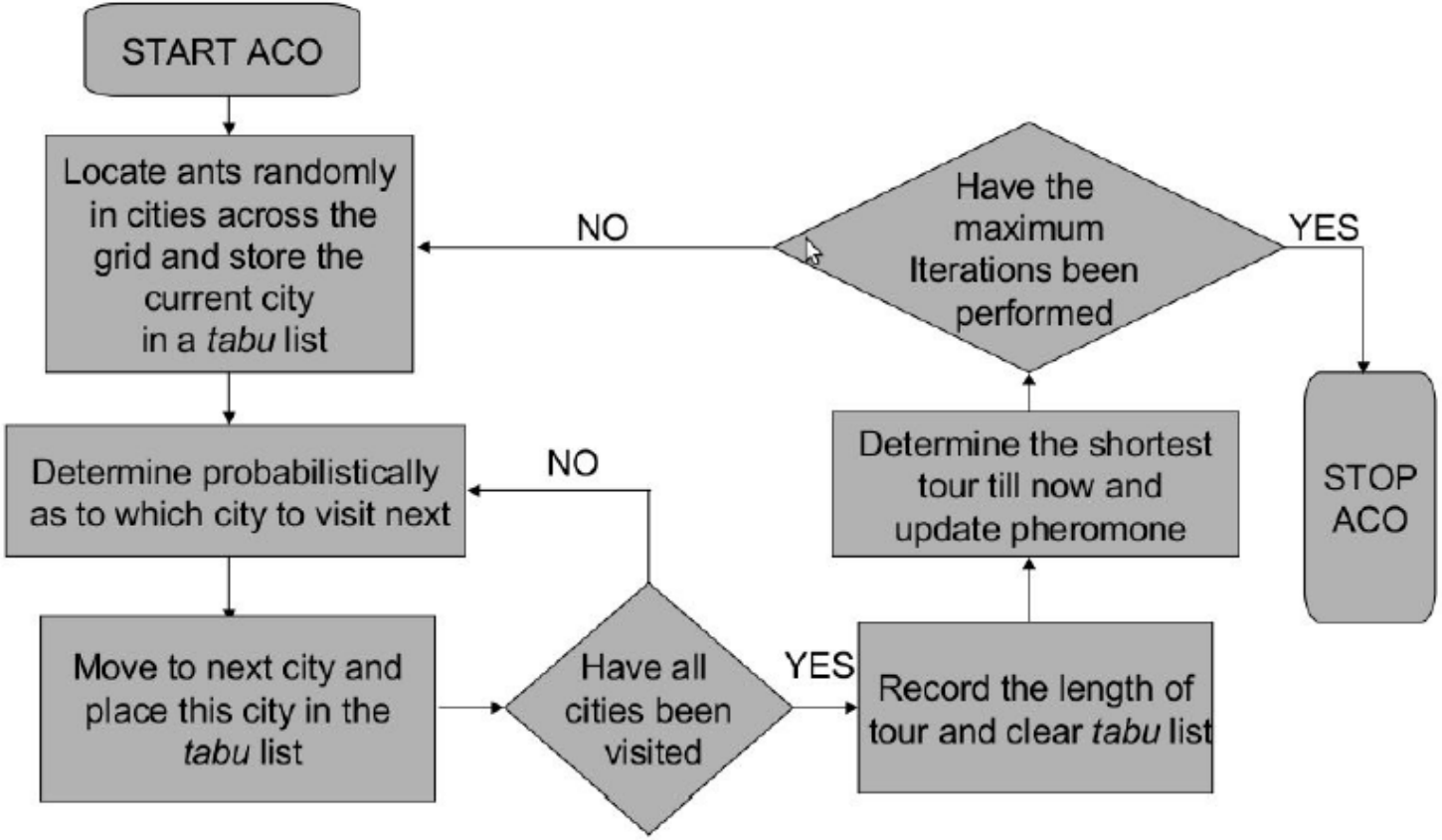
Example: Travelling Salesman Problem

- **Objective**: Find the shortest possible route that visits a given set of cities exactly once and returns to the starting city.

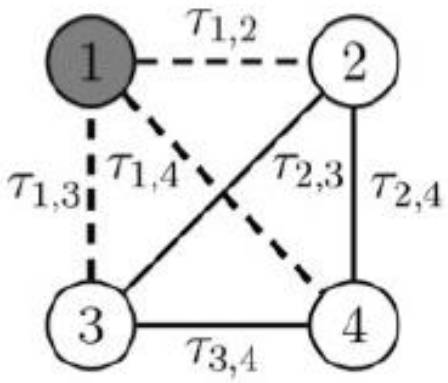
- **Input**: A list of cities and the distances between each pair of cities.

- **Output**: The shortest possible route that visits each city once and returns to the starting city.

Example: Travelling Salesman Problem

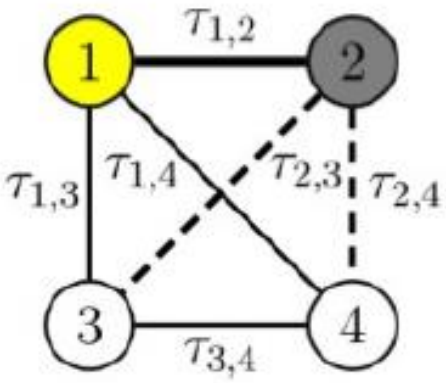


Example: Travelling Salesman Problem



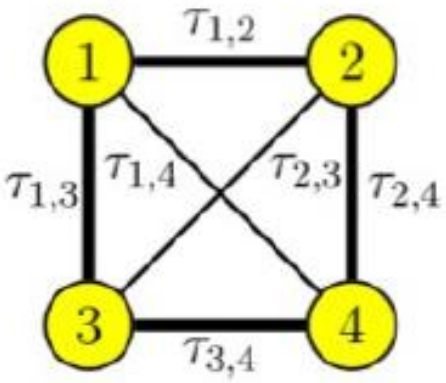
$$\mathbf{p}(e_{1,j}) = \frac{\tau_{1,j}}{\tau_{1,2} + \tau_{1,3} + \tau_{1,4}}$$

(a) First step of the solution construction.



$$\mathbf{p}(e_{2,j}) = \frac{\tau_{2,j}}{\tau_{2,3} + \tau_{2,4}}$$

(b) Second step of the solution construction.



(c) The complete solution after the final construction step.

TSP with SACO

```python
import numpy as np

# Calculate the distance matrix
def calculate_distance_matrix(num_cities, positions):
    distance_matrix = np.zeros((num_cities, num_cities))
    for i in range(num_cities):
        for j in range(num_cities):
            distance_matrix[i][j] = np.linalg.norm(positions[i] - positions[j])
    return distance_matrix

# Initialize pheromone matrix
def initialize_pheromone_matrix(num_cities, initial_pheromone_value=1.0):
    return np.full((num_cities, num_cities), initial_pheromone_value)

# Choose the next city based on the probability rule without heuristic information
def choose_next_city_saco(pheromone, current_city, visited):
    probabilities = []
    for city in range(len(pheromone)):
        if city not in visited:
            probability = pheromone[current_city][city]
            probabilities.append(probability)
        else:
            probabilities.append(0)
    probabilities = np.array(probabilities)
    probabilities /= np.sum(probabilities)
    next_city = np.random.choice(range(len(pheromone)), p=probabilities)
    return next_city
```

TSP with SACO

```python
# Run the SACO algorithm
def simple_ant_colony_optimization(num_cities, positions, num_ants, num_iterations):
    distances = calculate_distance_matrix(num_cities, positions)
    pheromone = initialize_pheromone_matrix(num_cities)
    best_path = None
    best_path_length = float('inf')

    for iteration in range(num_iterations):
        paths = []
        for ant in range(num_ants):
            path = [np.random.randint(num_cities)]
            while len(path) < num_cities:
                next_city = choose_next_city_saco(pheromone, path[-1], path)
                path.append(next_city)
            paths.append(path)

        for path in paths:
            path_length = sum([distances[path[i]][path[i + 1]] for i in range(len(path) - 1)])
            path_length += distances[path[-1]][path[0]]  # Closing the loop
            if path_length < best_path_length:
                best_path_length = path_length
                best_path = path

    best_path = [int(city) for city in best_path] # Convert np.int64 to Python int
    return best_path, best_path_length
```

TSP with SACO

```python
# Example usage
num_cities = 10
positions = np.random.rand(num_cities, 2)
num_ants = 5
num_iterations = 100

best_path, best_path_length = simple_ant_colony_optimization(num_cities, positions, num_ants,
num_iterations)

print("Best path:", best_path)
print("Best path length:", best_path_length)
```

```
Best path: [6, 3, 7, 4, 9, 1, 0, 2, 5, 8]
Best path length: 3.5225123117929926
```

ACO Algorithm

```
t = 0; Initialize parameters β, ρ1, ρ2, τ0, τ0, nk;
Place all ants, k = 1, ..., nk;
for each link (i, j) do
    τij(t) ~ U(0, τ0);
end
x̂(t) = ∅;
f(x̂(t)) = 0;
repeat
    for each ant k = 1, ..., nk do
        xk(t) = ∅;
        repeat
            if ∃j ∈ candidate list then
                Choose j ∈ Nik(t) from candidate list using equations (17.17) and
                (17.18);
            end
            else
                Choose non-candidate j ∈ Nik(t);
            end
            xk(t) = xk(t) ∪ {(i, j)};
            Apply local update using equation (17.21);
        until full path has been constructed;
        Compute f(xk(t));
    end
```

ACO Algorithm

$x = x^k(t) : f(x^k(t)) = \min_{k'=1,\ldots,n_k}\{f(x^k\ (t))\};$
Compute $f(x)$;
if $f(x) < f(\hat{x}(t))$ **then**
  $\hat{x}(t) = x$;
  $f(\hat{x}(t)) = f(x)$;
end
for *each link* $(i, j) \in \hat{x}(t)$ **do**
  Apply global update using equation (17.19);
end
for *each link* (i, j) **do**
  $\tau_{ij}(t+1) = \tau_{ij}(t)$;
end
$\hat{x}(t+1) = \hat{x}(t)$;
$f(\hat{x}(t+1)) = f(\hat{x}(t))$;
$t = t + 1$;
until *stopping condition is true*;
Return $\hat{x}(t)$ as the solution;

ACO Algorithm

$$j = \begin{cases} argmax_{u \in N_i^k(t)} \tau_{iu}(t)\eta_{iu}^{\beta}(t) & \text{if } r \leq r_0 \\ J & \text{if } r > r_0 \end{cases}$$

$$p_{iJ}^k(t) = \frac{\tau_{iJ}(t)\eta_{iJ}^{\beta}(t)}{\sum_{u \in N_i^k} \tau_{iu}(t)\eta_{iu}^{\beta}(t)}$$

$$\tau_{ij}(t+1) = (1-\rho_1)\tau_{ij}(t) + \rho_1 \Delta\tau_{ij}(t)$$

$$\Delta\tau_{ij}(t) = \begin{cases} \frac{1}{f(x^+(t))} & \text{if } (i,j) \in x^+(t) \\ 0 & \text{otherwise} \end{cases}$$

SACO

```python
import numpy as np

# Calculate the distance matrix
def calculate_distance_matrix(num_cities, positions):
    distance_matrix = np.zeros((num_cities, num_cities))
    for i in range(num_cities):
        for j in range(num_cities):
            distance_matrix[i][j] = np.linalg.norm(positions[i] - positions[j])
    return distance_matrix

# Initialize pheromone matrix
def initialize_pheromone_matrix(num_cities, initial_pheromone_value=1.0):
    return np.full((num_cities, num_cities), initial_pheromone_value)

# Choose the next city based on the probability rule with a factor q0
def choose_next_city_acs(pheromone, distances, current_city, visited, alpha, beta, q0):
    unvisited = [city for city in range(len(pheromone)) if city not in visited]
    if np.random.rand() < q0:
        probabilities = [(pheromone[current_city][city] ** alpha) * ((1 / distances[current_city]
[city]) ** beta) for city in unvisited]
        next_city = unvisited[np.argmax(probabilities)]
    else:
        probabilities = []
        for city in unvisited:
            probability = (pheromone[current_city][city] ** alpha) * ((1 /
distances[current_city][city]) ** beta)
            probabilities.append(probability)
        probabilities = np.array(probabilities)
        probabilities /= np.sum(probabilities)
        next_city = np.random.choice(unvisited, p=probabilities)
    return next_city
```

SACO

```
# Local pheromone update
def local_pheromone_update(pheromone, city_i, city_j, rho, initial_pheromone_value):
    pheromone[city_i][city_j] = (1 - rho) * pheromone[city_i][city_j] + rho *
initial_pheromone_value

# Global pheromone update
def global_pheromone_update(pheromone, best_path, distances, rho, Q):
    pheromone *= (1 - rho)
    for i in range(len(best_path) - 1):
        pheromone[best_path[i]][best_path[i + 1]] += rho * (Q / distances[best_path[i]]
[best_path[i + 1]])
    pheromone[best_path[-1]][best_path[0]] += rho * (Q / distances[best_path[-1]][best_path[0]])
 # Closing the loop
```

SACO

```python
# Run the ACS algorithm
def ant_colony_system(num_cities, positions, num_ants, num_iterations, alpha, beta, rho, Q, q0):
    distances = calculate_distance_matrix(num_cities, positions)
    pheromone = initialize_pheromone_matrix(num_cities)
    initial_pheromone_value = 1.0 / (num_cities * np.mean(distances))
    best_path = None
    best_path_length = float('inf')

    for iteration in range(num_iterations):
        paths = []
        for ant in range(num_ants):
            path = [np.random.randint(num_cities)]
            while len(path) < num_cities:
                current_city = path[-1]
                next_city = choose_next_city_acs(pheromone, distances, current_city, path, alpha,
beta, q0)
                path.append(next_city)
                local_pheromone_update(pheromone, current_city, next_city, rho,
initial_pheromone_value)
            paths.append(path)

        for path in paths:
            path_length = sum([distances[path[i]][path[i + 1]] for i in range(len(path) - 1)])
            path_length += distances[path[-1]][path[0]]  # Closing the loop
            if path_length < best_path_length:
                best_path_length = path_length
                best_path = path

        global_pheromone_update(pheromone, best_path, distances, rho, Q)

    best_path = [int(city) for city in best_path]  # Convert np.int64 to Python int
    return best_path, best_path_length
```

SACO

```python
# Example usage
num_cities = 10
positions = np.random.rand(num_cities, 2)
num_ants = 5
num_iterations = 100
alpha = 1.0
beta = 5.0
rho = 0.1
Q = 100
q0 = 0.9

best_path, best_path_length = ant_colony_system(num_cities, positions, num_ants, num_iterations,
alpha, beta, rho, Q, q0)

print("Best path:", best_path)
print("Best path length:", best_path_length)
```

```
Best path: [7, 3, 4, 2, 0, 8, 9, 1, 5, 6]
Best path length: 2.8996531288724943
```

SACO vs ACO

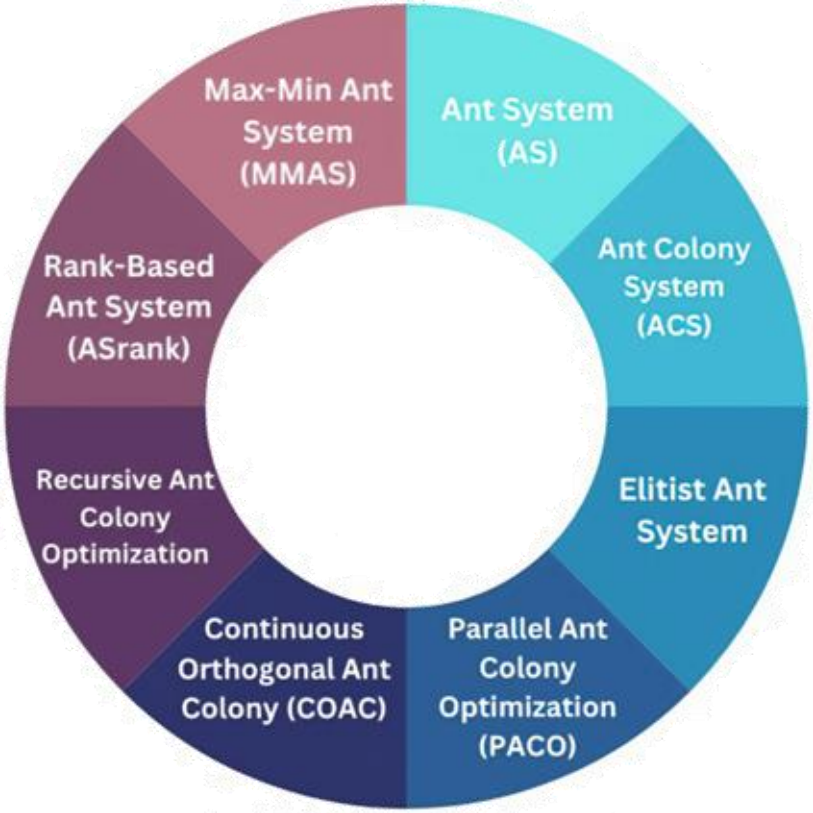
SACO

```
Best path: [6, 3, 7, 4, 9, 1, 0, 2, 5, 8]
Best path length: 3.5225123117929926
```

ACO

```
Best path: [7, 3, 4, 2, 0, 8, 9, 1, 5, 6]
Best path length: 2.8996531288724943
```

Variation of ACO



OUTLINES

Advantages & Disadvantages

Advantages & Disadvantages

- Advantages
  - Can be used in **dynamic** applications
  - **Positive Feedback** leads to rapid discovery of good solutions
  - **Distributed computation** avoids premature convergence
- Disadvantages
  - Convergence is **guaranteed**, but **time** to convergence **uncertain**
  - Coding is **not straightforward**

Discussion...

Q n A